Name of the Student	**Yoga Madha A**
Reg. No	**192425058**
GitHub Link	**https://github.com/YOGAMADHA/MLA0405-DEEP-LEARNING**

SIMATS ENGINEERING
CO3 - ASSESSMENT TOOL 2
Game Based Learning

COURSE NAME: DEEP LEARNING **COURSE CODE: MLA0405**
SLOT : D

COURSE OUTCOME COVERED

CO3: Analyze and apply convolutional neural network architectures, including convolutional layers, filters, parameter sharing, regularization, AlexNet and ResNet, to develop solutions for image classification and real-world computer vision applications. (L4)

CO Weightage: 25% **Duration: 1 Hour**

Assessment Tool Weightage: 35% **Marks: 50**

Code of Conduct:

I **Yoga Madha A-192425058** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Name of the student:**

Criteria	Understanding of Concept **(2.5)**	Conceptual Clarity **(2.5)**	Problem-Solving Ability **(2)**	Solution Design & Application **(2)**	Teamwork & Game Participation ***(1)***	**Total (10)**
Weightage	***25 %***	***25%***	***20%***	***20 %***	***10 %***	**100%**
Round 1						
Round 2						
Total						

Signature of the Faculty: ***Total Marks :***

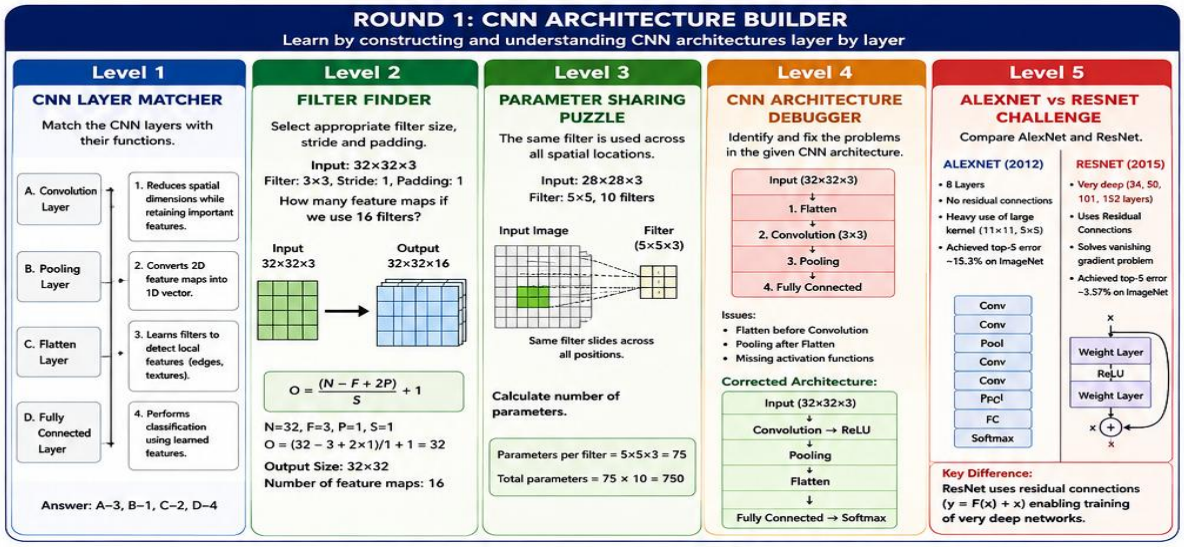
QUESTIONS:01 **Total Marks: 50**

CNN Game-Based Learning Activity – Completed Solution

Introduction

The **CNN Game-Based Learning Activity** is designed as a two-round learning game in which participants progressively develop their understanding of Convolutional Neural Networks. Each round contains five levels. The challenges begin with basic CNN concepts and gradually move toward architecture design, feature extraction, regularization, architecture comparison, and real-world applications.

The activity helps learners understand not only the theoretical concepts of CNNs but also how these concepts are applied when designing an actual image-classification system.



ROUND 1 – CNN Architecture Builder

Level 1 – CNN Layer Matcher

The first challenge is to match each CNN layer with its correct function.

CNN Layer	Function
Convolution Layer	Applies learnable filters to detect local features such as edges, textures, and shapes.
Pooling Layer	Reduces the spatial dimensions of feature maps while retaining important information.
Flatten Layer	Converts the multidimensional feature maps into a one-dimensional vector.
Fully Connected Layer	Uses the extracted features to perform final classification.

Correct Matching

A – 3

B – 1

C – 2

D – 4

Explanation

The **convolution layer** is responsible for feature extraction. Filters move across the image and detect useful patterns. The **pooling layer** performs dimensionality reduction, which decreases computational cost and helps make the network less sensitive to small positional changes.

After convolution and pooling, the **flatten layer** converts the feature maps into a vector. This vector is passed to the **fully connected layer**, which learns relationships between the extracted features and the target classes.

Level 2 – Filter Finder
A convolution filter is a small matrix of weights that moves across an image to detect specific patterns.
For example, consider:

- Input size = **32 × 32 × 3**
- Filter size = **3 × 3**
- Padding = **1**
- Stride = **1**
- Number of filters = **16**

The output-size formula is:

$$O = \frac{N - F + 2P}{S} + 1$$

Substituting the values:

$$O = \frac{32 - 3 + 2(1)}{1} + 1$$

$$O = 32$$

Therefore, each filter produces a:

32*32

feature map.
Since there are **16 filters**, the final output is:

$$\boxed{32 \times 32 \times 16}$$

Answer
Number of feature maps = 16
Each filter learns to identify a different type of feature. For example, one filter may detect vertical edges, another horizontal edges, and others may learn textures or more complex patterns.
Effect of Filter Parameters
Filter size: A larger filter can observe a larger region of the image but requires more parameters.
Stride: Increasing stride makes the filter move in larger steps, reducing the output spatial dimensions.
Padding: Padding adds pixels around the input and helps preserve spatial dimensions.

Level 3 – Parameter Sharing Puzzle
Consider:

- Input = (28\times28\times3)
- Filter = (5\times5)
- Number of filters = 10

Each filter must operate across all three input channels.
Therefore, the number of weights in one filter is:

5×5×3=75

For 10 filters:

$$75 \times 10 = 750$$

If each filter has one bias:

$$750 + 10 = 760$$

Parameter Sharing

Parameter sharing means that the **same filter weights are reused at different spatial positions** of an image.

For example, if a filter learns to detect a vertical edge, the same filter can detect that vertical edge whether it occurs on the left, center, or right side of the image.

This greatly reduces the number of parameters compared with a fully connected network.

Importance

Parameter sharing:

- Reduces memory requirements.
- Reduces computational complexity.
- Allows the network to detect the same feature at different locations.
- Makes CNNs especially suitable for image processing.

Level 4 – CNN Architecture Debugger

Suppose the incorrect architecture is:

Input→Flatten→Convolution→Pooling→Fully Connected

This architecture contains an important problem because the **Flatten layer appears before the convolution layer**.

The corrected architecture is:

$$Input \rightarrow Convolution \rightarrow ReLU \rightarrow Pooling \rightarrow Flatten \rightarrow Fully\ Connected \rightarrow Softmax$$

Why is this correct?

The input image must first pass through convolution layers so that the CNN can extract spatial features.

The **ReLU activation** introduces non-linearity and helps the network learn complex relationships.

The **pooling layer** reduces spatial dimensions.

Only after feature extraction should the feature maps be flattened into a vector and supplied to the fully connected layer.

Main Errors Identified

1. Flatten was placed before convolution.
2. Pooling was incorrectly positioned.
3. Activation function was missing.
4. The architecture did not follow the normal feature-extraction-to-classification flow.

Correct Architecture

Input Image
↓
Convolution
↓
ReLU
↓
Pooling
↓
Convolution
↓
ReLU

↓

Pooling

↓

Flatten

↓

Fully Connected

↓

Softmax

↓

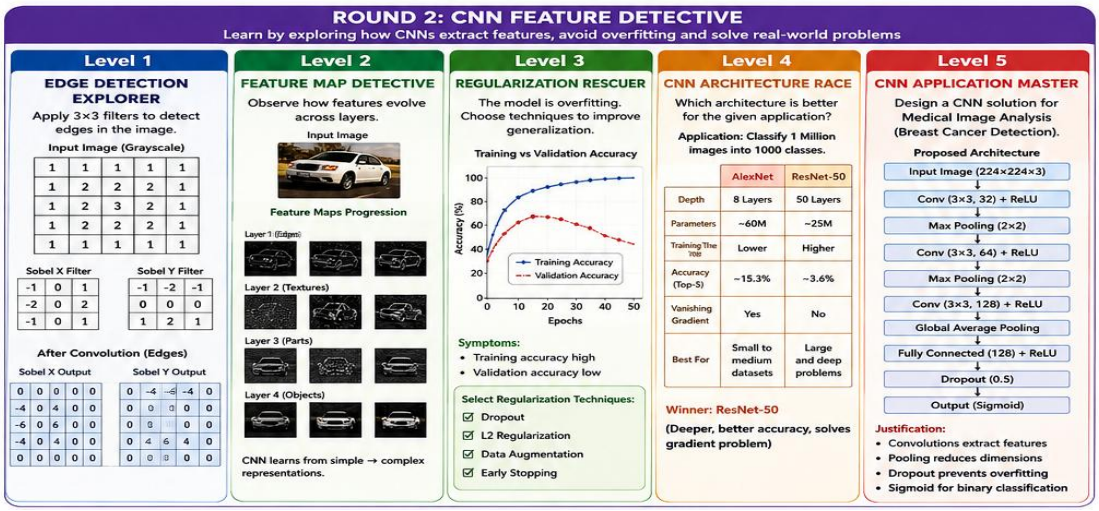
Class Prediction

Level 5 – AlexNet vs ResNet Challenge

Comparison

Feature	**AlexNet**	**ResNet**
Introduction	2012	2015
Main idea	Deep CNN with convolution and pooling	Very deep CNN using residual learning
Architecture	Conventional sequential CNN	Uses residual/skip connections
Skip connections	No	Yes
Main problem addressed	Image classification performance	Vanishing/degradation problem in deep networks
Depth	8 learned layers	Variants such as ResNet-18, 34, 50, 101, 152
Activation	ReLU	ReLU
Major contribution	Demonstrated the power of deep CNNs	Enabled effective training of very deep networks

ROUND 2 – CNN Feature Detective



Level 1 – Edge Detection Explorer

A CNN can use small filters to detect edges.

Two common (3\times3) Sobel filters are:

Horizontal/Vertical Edge Filters

$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

and

$$S_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

The filter is placed over a small region of the image and the corresponding values are multiplied and added.

The general convolution operation is:

$$Output(i,j) = \sum_m \sum_n I(i+m, j+n) K(m,n)$$

where:

- (I) = input image
- (K) = convolution kernel
- (i,j) = current position

Interpretation

If the convolution produces a large positive or negative value, there is likely to be a strong edge at that location.

Thus, convolution allows CNNs to identify:

- Vertical edges
- Horizontal edges
- Diagonal edges
- Corners
- Textures

The first CNN layers normally learn simple features such as these.

Level 2 – Feature Map Detective

A CNN does not immediately recognize a complete object. It gradually builds a representation through multiple layers.

Feature Progression

Input Image
↓
Early CNN Layers
↓
Edges
↓
Textures
↓
Shapes / Parts
↓
Object Components
↓
Complete Object
↓
Classification

Layer 1 – Simple Features
The first convolutional layers usually detect:

- Edges
- Lines
- Corners
- Basic color patterns

Middle Layers – Intermediate Features
The middle layers combine these simple features to detect:

- Textures
- Curves
- Shapes
- Object parts

Deep Layers – Complex Features
The deeper layers combine intermediate features to recognize:

- Faces
- Cars
- Animals
- Tumors
- Traffic signs
- Other complete objects

Level 3 – Regularization Rescuer
Suppose a CNN achieves:

- Training accuracy = **98%**
- Validation accuracy = **70%**

This indicates that the model is performing extremely well on training data but poorly on unseen data.
Suitable Solutions
1. Dropout
Randomly disables some neurons during training.
For example:

Dropout=0.5

means approximately 50% of the selected neurons are randomly dropped during each training step.
2. L2 Regularization
Adds a penalty for large weights:

$$Loss_{total} = Loss_{original} + \lambda \sum w^2$$

This encourages the network to maintain smaller weights.
3. Data Augmentation
Creates modified training examples using operations such as:

- Rotation
- Cropping
- Flipping
- Scaling
- Translation

This increases training-data diversity.
4. Early Stopping
Training is stopped when validation performance stops improving.
Best Strategy

A practical solution is:

Data Augmentation + Dropout + L2 Regularization + Early Stopping

These techniques improve the model's ability to generalize to unseen images.

Level 4 – CNN Architecture Race

Suppose two architectures are being considered for a large image-classification problem.

AlexNet

AlexNet is relatively simpler and shallower. It is useful for understanding the basic CNN architecture and can work well for smaller or less demanding problems.

ResNet

ResNet is deeper and uses residual connections. It is particularly useful when the problem requires learning complex visual representations.

Requirement	**Better Choice**
Simple CNN experiment	AlexNet
Small/medium image dataset	AlexNet or a smaller ResNet
Very complex image recognition	ResNet
Very deep architecture	ResNet
Difficult gradient flow	ResNet
Large-scale classification	ResNet
Learning residual representations	ResNet

Justification

ResNet can contain many layers while maintaining effective gradient flow through residual connections. This allows it to learn more complex representations than a relatively shallow architecture such as AlexNet.

Level 5 – CNN Application Master

Application: Medical Image Analysis

A practical application of CNNs is **medical image analysis**, such as detecting abnormalities in X-ray, CT, MRI, or other medical images.

Proposed Architecture

Medical Image
↓
Image Preprocessing
↓
Convolution 3×3 + ReLU
↓
Max Pooling 2×2
↓
Convolution 3×3 + ReLU
↓
Max Pooling 2×2
↓
Convolution 3×3 + ReLU
↓
Global Average Pooling
↓
Fully Connected Layer
↓

Dropout

↓

Output Layer

↓

Normal / Abnormal

Layer Functions

Input Layer: Receives the medical image.

Convolution Layers: Extract edges, textures, shapes, and abnormal visual patterns.

ReLU: Introduces non-linearity and allows the network to learn complex patterns.

Max Pooling: Reduces spatial dimensions and computational requirements.

Global Average Pooling: Converts feature maps into a compact representation while reducing the number of parameters.

Fully Connected Layer: Combines the extracted features for classification.

Dropout: Reduces overfitting.

Output Layer: Produces the final prediction.

For binary classification, a sigmoid function can be used:

$$P(y=1)=\sigma(z)$$

where

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

A **ResNet-based architecture** would be a strong choice for a complex medical-image classification task because residual connections allow the use of deeper networks while helping maintain effective gradient flow.

However, medical applications require careful validation because incorrect predictions can have serious consequences. CNN predictions should therefore support, rather than automatically replace, qualified medical professionals.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Conceptual Clarity	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Problem-Solving Ability	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design & Application	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Teamwork & Game Participation	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand